

DepFiN: A 12-nm Depth-First, High-Resolution CNN Processor for IO-Efficient Inference

Koen Goetschalckx¹, Fengfeng Wu², and Marian Verhelst¹, *Senior Member, IEEE*

Abstract—Applying convolutional neural networks (CNNs) on high-resolution images leads to very large intermediate feature maps (FMs), which dominate the memory traffic. Processing in the classical layer-by-layer order creates the requirement to store the complete FMs at once, when moving from one layer to the next. As the size of these FMs only realistically allows this in off-chip memory, this leads to high off-chip bandwidth, which comes at great energy costs. The DepFiN processor chip, presented in this article, overcomes this cost by running CNNs in a deep layer fusion mode, dubbed depth-first execution, made possible by a control flow that supports frequently switching between layers. To furthermore tackle the computational cost as well, the computationally efficient depthwise + pointwise (DW + PW) layer pairs are explicitly supported in DepFiN by a novel accelerator core that can dynamically change its configuration to manage the low computational intensity of the depthwise layers. Benchmarking measurements show the 12-nm DepFiN chip reaching up to 20 TOPS/W peak, 8.2 TOPS/W on the MC-CNN-fast stereo-matching network excluding input-output (IO) power (at 8-bit 0.6 V_{dd}) and, crucially, 3.95 TOPS/W with the IO power included on the same network and an up to 18× improvement realized by supporting depth-first (MC-CNN-fast at 8-bit, 0.65 V_{dd}).

Index Terms—Accelerator, convolutional neural network (CNN), depth-first, high-resolution, layer fusion, pixel processing.

I. INTRODUCTION

CONVOLUTIONAL neural networks (CNNs) have realized state-of-the-art quality on image processing tasks such as computational photography, demosaicing, super-resolution, super slow motion, denoising, and depth extraction [1], [2], [3], [4], [5], [6]. In contrast to image classification, which typically operates on relatively small images (e.g., 224×224 pixels [7]) to create a single number output (the detected class), these *pixel processing* tasks are preferably performed on high-resolution inputs and, moreover, have similarly large outputs as well. This leads to such an increase in the size of intermediate feature maps (FMs) that

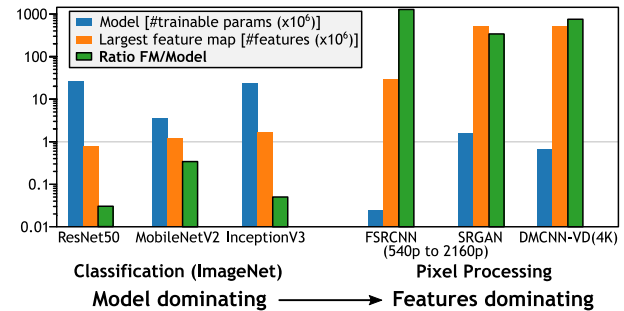


Fig. 1. In pixel processing CNNs, FMs dominate the memory usage instead of the CNNs' models.

these can be orders of magnitude larger than the CNNs' model size, as illustrated in Fig. 1.

Traditionally, CNN inference is done in a layer-by-layer fashion: a layer is processed completely, generating its full output FM, before that output is used to start processing a subsequent layer in the network. Executing inference this way on pixel processing CNNs requires either a high amount of transfers of large intermediate FMs to and from the accelerator [external input-output (IO)] or very large on-chip feature memories. The former is especially costly in terms of energy, while the latter requires a large amount of die area. Table I illustrates this quantitatively for several simple feedforward networks, each consisting of either ten layers of 3×3 Conv3D filters, or ten layer pairs (20 layers total) of 3×3 depthwise + pointwise (DW + PW) filters, executed on $1280 \times 720 \times 3$ images. The different lines in the table relate to different numbers of channels in the intermediate FMs of the networks under study and to different available on-chip memory capacities assumed in the system. For each network–system combination, an optimistic analytical model [8] optimizes which data can remain in the on-chip buffers, and which have to be pushed out to external memory to be refetched later. This allows to compute the required external IO bandwidth and its energy for each combination, resulting in a maximum achievable energy efficiency when solely considering this IO bottleneck. As can be seen, the maximum achievable computational energy efficiency (in TOPS/W) for such pixel processing networks ranges from very low to mediocre, while only external IO energy cost is taken into account, ignoring any on-chip processing or memory costs. Also shown in the table is that the achievable throughput (in TOPS) and efficiency (in TOPS/W) of the DW + PW are always lower than for the comparable Conv3D network, because the former has twice the number of layers and thus requires approximately twice the amount of IO. As such, instead of achieving energy savings with DW + PW layers by the decrease in the number

Manuscript received 8 March 2022; revised 21 June 2022 and 29 August 2022; accepted 24 September 2022. Date of publication 14 October 2022; date of current version 25 April 2023. This article was approved by Associate Editor Meng-Fan Chang. This work was supported in part by KU Leuven; in part by the Flemish Government (Artificial Intelligence (AI) Research Program), KU Leuven; in part by the European Union European Research Council (EU ERC) Project Re-SENSE under Grant ERC-2016-STG-715037; and in part by Huawei Technologies France Simplified Joint Stock Company [société par actions simplifiée unipersonnelle (SASU)]. (Corresponding author: Koen Goetschalckx.)

Koen Goetschalckx and Marian Verhelst are with the MICAS Group, Department of Electrical Engineering (ESAT), KU Leuven, 3001 Leuven, Belgium (e-mail: kgoetsch@esat.kuleuven.be; marian.verhelst@kuleuven.be).

Fengfeng Wu was with GlobalFoundries, Beijing 100190, China. He is now with Product Research and Development Department, Witmem, Beijing 100080, China (e-mail: fengfengwu@yeah.net).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/JSSC.2022.3210591>.

Digital Object Identifier 10.1109/JSSC.2022.3210591

TABLE I
EXAMPLE MAXIMUM LAYER-BY-LAYER PERFORMANCE CONSTRAINED
ONLY BY EXTERNAL IO AND ON-CHIP MEMORY CAPACITY

Architecture	#channels	Set available on-chip feature memory [MiB]	Feature IO [gigabits] ⁽¹⁾	Energy ^(1,2) of feature IO [J]	max. TOPS ^(1,3)	max. TOPS/W given IO E. ^(1,2)
Conv3D	16	0	2.1	0.04	2.3	0.84
Conv3D	16	2	1.8	0.04	2.7	0.97
Conv3D	16	8	0.9	0.02	5.3	1.93
Conv3D	32	0	4.3	0.09	4.5	1.64
Conv3D	32	2	4.0	0.08	4.8	1.76
Conv3D	32	8	3.0	0.06	6.2	2.28
dw-pw	16	0	4.3	0.09	0.2	0.07
dw-pw	16	2	3.7	0.07	0.2	0.09
dw-pw	16	8	1.8	0.04	0.5	0.17
dw-pw	32	0	8.5	0.17	0.3	0.12
dw-pw	32	2	7.9	0.16	0.4	0.13
dw-pw	32	8	6.1	0.12	0.5	0.17

Assuming ⁽¹⁾: optimistic assumptions from [8] with layer-by-layer execution; ⁽²⁾: 20pJ/transfered bit [9]; ⁽³⁾: 17GBps external IO bandwidth [10]; a 10×Conv3D or 10×(depthwise+pointwise) layer network with 3×3 kernels and 1280×720×3 inputs and outputs

of operations required, this change from Conv3D to DW + PW layers actually decreases the efficiency because it is held back even more by the memory hierarchy.

To alleviate the effect of these large FMs on the memory hierarchy, one can resort to the compression techniques [11], possibly helped by the sparsity increasing methods [12] and (aggressive) quantization [13]. However, while these techniques help, they often do not provide enough savings as activations are not sparse enough, or they can some of the network's accuracy.

Alternatively, one can use depth-first [8] execution instead of layer-by-layer CNN inference. However, this impacts parallelization opportunities and as such complicates maintaining a high utilization across processors consisting of many processing elements (PEs), which is required for high throughput. This article, therefore, presents a parallelization-friendly variant of the *depth-first* method [8] in Section II and describes how it is implemented in a compatible accelerator called DepFiN [14] at both the hardware-level control and the software-level scheduler/compiler flow (see Section V).

With depth-first inference tackling the external IO limitation, DW + PW pairs can be used in the CNN's architecture without trashing system-level energy consumption due to extensive usage of external IO. However, the computational intensity for the DW layers is so low that even on-chip memory bandwidth is insufficient to fully accelerate them. To overcome this, DepFiN introduces a novel core that can operate in a pipelined DW + PW configuration with one part of the array processing the DW layer and feeding the outputs directly to the other part of the array computing the subsequent PW layer to save the bandwidth of the on-chip feature memory, much like how depth-first propagates data sooner to avoid off-chip traffic. This special configuration both overcomes the need to parallelize the DW layer further to achieve good core utilization and decreases the on-chip static random access memory (SRAM) usage.

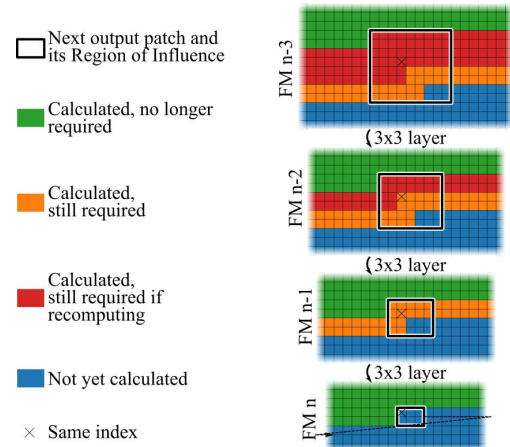


Fig. 2. Example ROI period and status of pixels across layers. Storing a few line buffers (orange) in every layer avoids superlinear memory increase with added layers.

The highlights of this article are as follows:

- 1) we **show the need for depth-first inference** and its benefits in the external IO versus on-chip memory capacity trade-off space (see Section I);
- 2) we present **parallelization-friendly adaptation** of this depth-first inference (see Section II);
- 3) an application-specific instruction set processor (ASIP) implementation, DepFiN, with a **control flow compatible with the frequent layer switching** required to operate in a depth-first fashion (see Sections III and V);
- 4) its **novel core that can dynamically change its topology** to achieve high utilization for both Conv3D and the computationally efficient DW + PW (see Section IV); and
- 5) **benchmark** it on synthetic CNNs and CNNs from literature of up to 20 layers deep,¹ consisting of both Conv3D and DW + PW layers, all of which DepFiN runs **fully fused** to realize **zero external IO for intermediate features** for up to 18× better system efficiency (see Section VI).

II. DEPTH-FIRST PROCESSING

The traditional layer-by-layer processing requires to keep a record of the complete intermediate FMs when moving from one layer to the next. This can be analyzed by assessing the *lifetime* of the individual features in the FMs: the time between their generation and their last usage. This parameter is very large when layers are processed one after the other, because the first *pixels*² of an FM are only consumed after the complete FM has been calculated and the next layer's execution starts. Various layer fusion methods try to reduce this pixel lifetime, by already doing all the computations that consume the first pixels before generating the rest of the FM. In practice, this means that a layer L_i already starts processing the FM FM_{i-1} generated by layer L_{i-1} as soon as there are enough data to generate some features in FM_i . Applying

¹“Deep” and “depth” in this article relate to the position of an FM in a CNN, not to the number of channels a layer has.

²In this article, we use *pixel* to describe the feature data belonging to one spatial X–Y location of an FM across all different channels.

this to all the layers makes it so that new input pixels of the network propagate through all the layers of the network, generating new pixels in the FMs of all layers where sufficient input data are available, before the next new input pixel is required. In other words, the sliding windows of all the layers move together in lockstep, instead of each one only starting when the previous one finished its full sweep. In this way, intermediate FMs do not need to be stored as a whole. This avoids the need to store intermediate features in high-capacity, external memories, and as such saves the external bandwidth normally needed to communicate these intermediate features to the external memories. This is the main idea behind depth-first [8], or deeply fused layer, processing of CNNs.

To make a practical implementation of this idea, one can construct a region-of-influence (ROI) pyramid from a certain patch in the output of the last layer of the stack, by tracing back through the network which pixels are required to compute it. Fig. 2 shows such pyramid (thicker black boundaries), traced back from a 2×3 output patch over three 3×3 layers. Note that the channel dimension is omitted in this illustration. If we assume the calculation progresses through the FM row by row (left to right, followed by top-bottom), Fig. 2 indicates which pixels have been calculated previously (green, red, and orange) and are still-to-be-computed (in blue). Where the ROI pyramids of the already computed output patches (green) overlap with the ROI pyramids of still-to-compute output patches (blue), Fig. 2 uses a red or orange color. Several strategies can be applied to deal with these overlaps.

1) Recompute: First, one could recompute the pixels in this overlap every time they are needed [15], [16]. However, as the number of layers increases, the size of this overlap in each FM keeps increasing.

Therefore, the output patch size should be large enough to keep the overlap region small relative to the patch size, to prevent doing too much work more than once [16]. This strategy thus favors few layers and large output patches. Although this makes it close to the layer-by-layer paradigm in practice, it limits the required on-chip feature memory capacity to the largest patch in the pyramid instead of the largest complete FM.

2) Cache: Second, instead of recomputing them, one could cache the pixels of the overlap regions **off-chip** (for large patches with limited overlap), **on-chip** (for small patches [8], [15], [17]), or **both** [8], [18] (different memories for both window sweeping directions and/or extra levels of tiling).

Depth-first processing [8] aims at the caching approach to minimize the off-chip traffic, while optimally exploiting the available on-chip storage. To know how much on-chip storage is required to support this scheme, it is important to note that only already computed pixels that are within a single kernel distance of the still-to-be-computed pixels need to be retained for further processing (orange area in Fig. 2). The influence of all other pixels that are within the ROI of the output patch currently under processing (red pixels in Fig. 2) is already fully incorporated in the cached pixels of the next layer, and these pixels can hence be discarded. This is a key difference with the recompute strategy, which requires the full ROI at every layer to recompute the intermediate pixels.

As a result, the depth-first strategy needs to cache only a few FM rows per layer, colored orange in Fig. 2 and dubbed “*line buffers*,” depending on the filter size and stride of the current layer and independent of the network depth. This makes the memory capacity cost of stacking more layers linear to the number of layers for this technique as all the line buffers need to be stored simultaneously but their individual size is independent on the number of layers. In contrast, the memory capacity requirement for the recompute technique grows superlinearly with the number of layers like the ROI size.

It is also important to note that the patch of new intermediate pixels that need to be calculated in each intermediate layer is equal to the size of the targeted output patch (in regime and disregarding the channel dimension and strides). All other pixels required to generate the output patch, or a patch in any of the intermediate FMs, are already cached. This regularity of the patch dimensions can ease the design of efficient accelerators and their scheduling.

DepFiN uses patches of 1 pixel high and 128 pixels wide. This patch size helps parallelization and data management (see section IV-A1) while sticking close to the ideal depth-first granularity of 1 pixel. Note that each patch is too small to efficiently use graphics processing units (GPUs), while the total amount of computations is still too high to efficiently map to CPUs. Hence, a CNN accelerator that can work efficiently with small patches and can quickly alternate between several layers of a CNN is required for efficient depth-first processing. An overview of such an accelerator, dubbed *DepFiN*, is given in the next section and followed by a description of how it computes each of these patches. Section V details how a depth-first execution schedule, that is when each patch is fetched or computed, is created.

III. DEPFiN OVERVIEW

This section gives a high-level overview of the DepFiN accelerator and its programming flow, as shown in Fig. 3. The core of DepFiN consists of an array 16×128 PEs. This array computes patches of 128 horizontally neighboring pixels. It is sourced with features from an on-chip feature memory, the interface of which is elaborated on in Section IV-A1b). Every clock cycle, 16 weights are provided in parallel from an on-chip weight memory. Both memories consist of multiple banks of single-port SRAMs.

On a low level, the PE array, the memory addresses, the IO, etc. are all controlled by configurable finite state machines (FSMs). Each of these maps to a single task on a 1×128 pixels patch. Table II lists them with their descriptions and most important parameters.

As several FSMs can run concurrently—for instance, one is running a convolution on the array while another is loading data from external IO into the feature memory—they might try to access the single-port memories at the same time. Therefore, multiple SRAM banks are used, and access to them is controlled by the *concurrent access arbiter*, which regulates memory accesses to the FSMs based on fixed priorities.

The FSMs are instructed by programmable microprocessor (μ PC), which configures and starts them according to a

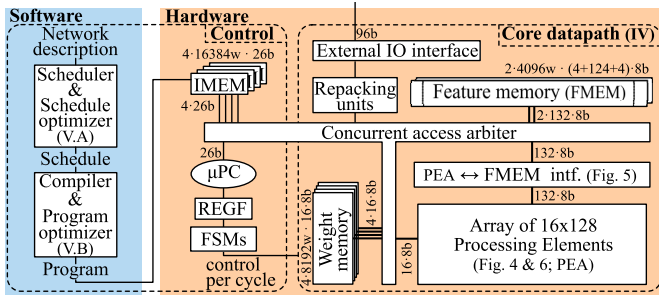


Fig. 3. DepFiN system overview.

TABLE II
FSMs IMPLEMENTED IN DEPFiN CONTROL HARDWARE

Conv3D	<i>Descr.:</i> Computes a patch of a Conv3D layer <i>Params:</i> Addresses, kernel dimensions, accumulation registers,...
DW+PW	<i>Descr.:</i> Computes a patch with pipelined 3×3 or 5×5 DW+PW <i>Params:</i> Addresses, kernel dimensions, accumulation registers to use, scaling & activation parameters of DW layer,...
Output array	<i>Descr.:</i> Systolically moves accumulation results to the feature memory, including scaling and activation <i>Params:</i> Accumulation registers, addresses, scaling & activation parameters, ...
Load	<i>Descr.:</i> Loads a patch from external IO link <i>Params:</i> #channels, addresses, ...
Send	<i>Descr.:</i> Sends a patch from feature memory over external IO link <i>Params:</i> #channels, addresses, ...
Load weights	<i>Descr.:</i> Loads weights from external IO link <i>Params:</i> Size, address, ...
Load program	<i>Descr.:</i> Loads program from external IO link <i>Params:</i> Size, address, ...

program stored in its instruction memory. The program is generated off-line by a scheduler–compiler flow (see Section V).

IV. RECONFIGURABLE CORE

This section describes the accelerator core of DepFiN, which consists of 2048 PEs that can be reconfigured on-the-fly into different configurations as follows.

- 1) A **base configuration**, meant for all plain Conv3D layers.
- 2) A **novel split configuration** where one part of the array computes a 3×3 depthwise layer, while the rest of the array immediately uses the output of that depthwise layer to calculate a subsequent pointwise layer.
- 3) A similar configuration for 5×5 depthwise layers.

All the configurations generate FM patches of 1×128 pixels large. Section IV-A describes top-level processing array architecture and functioning, while Section IV-B details the support for these configuration in the PEs. Finally, Section IV-C discusses two implemented power-saving techniques.

A. Array Organization and Configurability

1) Base Configuration [see Fig. 4(a)]:

a) *Data flow:* To keep the 2048 PEs used, a 2048-way parallelization is required. To provide this level of parallelism, DepFiN benefits from the targeted applications' high-resolution FMs by applying $128 \times$ spatial unrolling of the

horizontal FM dimension (O_x). One could opt to also use the vertical FM dimension (O_y). However, using both O_x and O_y complicates the data layout in memories, especially because of the overlap between ROIs of each patch. With only O_x parallelized, the feature memory and its interface as described below can be used to deal with the horizontal overlap between tiles, while vertical overlaps can be fully handled in software (see Section V).

The $2048/128 = 16 \times$ more parallelism is still required for maximal utilization. This is provided by spatially unrolling the output channel (OC) loop, because the spatial kernel dimensions (K_x and K_y) are too small and spatially unrolling the input channel (IC) or vertical FM (O_y) dimensions would require more feature memory bandwidth, which already has to support 128 features in parallel.

This results in a 16×128 array of 2048 PEs in total, in which 128 horizontally neighboring input features are broadcasted vertically (for the sake of visualization; red in Fig. 4) over 16 rows of PEs, and 16 equivalent weights for 16 different OCs are broadcasted horizontally (green in Fig. 4) over 128 PE columns. We further choose output stationarity to avoid extra bandwidth from/to on-chip feature memories. Fig. 4(a) shows this data flow.

b) *PE array ↔ feature memory interface* (see Fig. 5): In all, 128 features are fed into the PE array each cycle. To lower the average bandwidth from the on-chip memory required to supply these features, the 128 horizontally neighboring features already available are temporally reused by DepFiN's core for the K_x loop of the convolution. Several design aspects of the interface between the PE array and the feature memory (see Fig. 5) support this as follows.

- 1) The feature memory bandwidth is increased from 128 features in parallel to 132, so that two extra features are available on each side for support up to $K_x = 5$.
- 2) The feature memory is logically split into three parts of four features, 124 features and four features wide. A mux (side swapper) as such allows to avoid storing features at the boundary of 1×128 patches twice (once as a feature on the right and once as a feature on the left of a patch).
- 3) A row of registers is added to buffer these 132 features, so that the feature memory itself needs to be read only once for the whole K_x loop.
- 4) A feature shifter is added to generate the $-2/-1/0/+1/+2$ horizontally offset versions of the 128 features for the datapath.

The output patches are written back from the core to the three feature memory blocks with a proper alignment to serve as an input to the next layer (see Section V-A).

2) *DW + PW Configurations* [see Fig. 4(b)–(d)]: The data flow of the base configuration relies on the parallel computation of 16 OCs to occupy the 16 rows of PEs and the spatial reuse of input features across these 16 OCs. For depthwise layers, however, there is no reuse of features across OCs, which results in either large underutilization of the array or a large increase in required input bandwidth. Note that this is an inherent property of depthwise layers due to their low computational intensity. Another parallelization solution

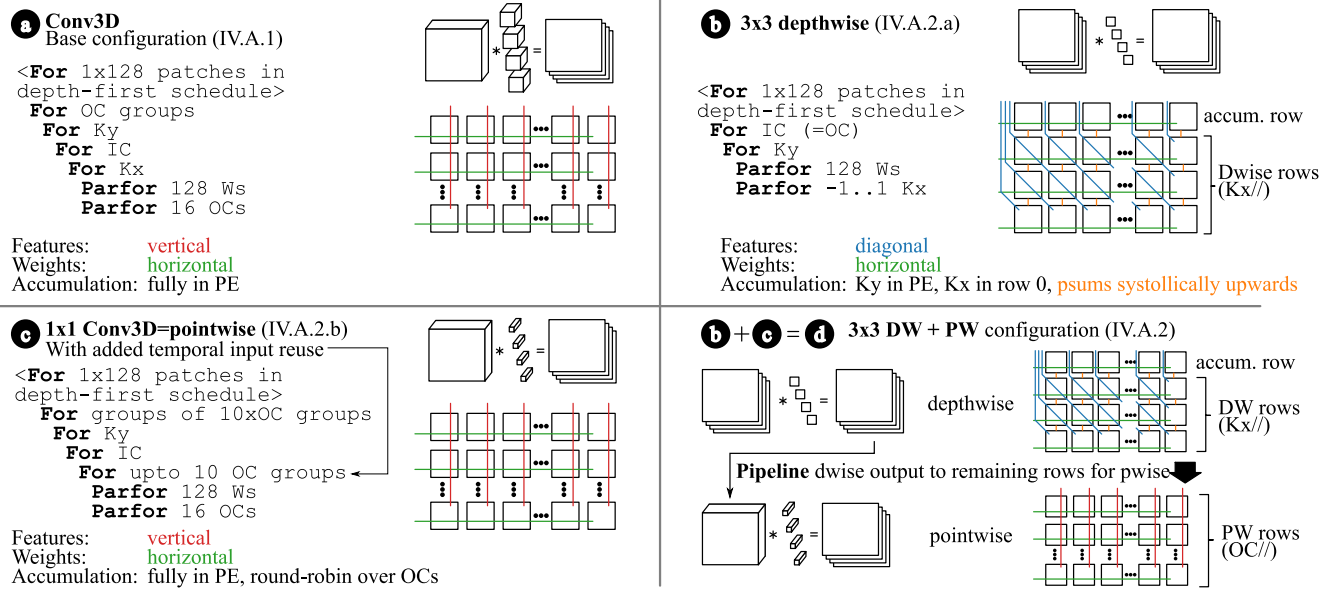


Fig. 4. (a) Base PE array configuration and (d) pipelined DW + PW configuration built from subconfigurations (b) and (c).

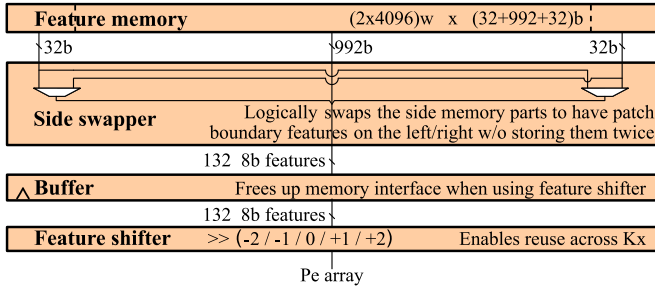


Fig. 5. Memory interface between the PE array and the feature memory interface [see Section IV-A1b)] efficiently supports Kx loops.

is necessary to increase utilization. However, as explained above for the base configuration, there are no more good parallelization candidates left. Therefore, DepFiN's core uses a novel two-part solution to still maintain a high utilization in the presence of depthwise layers as follows.

- 1) For the depthwise layer itself, parallelization of the horizontal spatial kernel dimension (Kx) is used [see Fig. 4(b)].
- 2) To use the remaining PE rows, the result from the depthwise layer is, without intermediate SRAM access, directly pipelined toward these rows, which calculates the subsequent pointwise layer. [see Fig. 4(c)].

This pipelining of two parts of the array can be viewed as an implementation of depth-first execution on the PE array, to avoid SRAM accesses (rather than external IO) as also in this case intermediate features are propagated sooner to avoid a large memory bandwidth. At the same time, it removes the requirement to parallelize the depthwise layer across the whole PE array to achieve good utilization thereof.

a) Depthwise part: To realize the parallelization of the horizontal filter dimension (Kx) for the 3×3 or 5×5 depthwise layer, features must be broadcasted diagonally across the second to sixth rows, and to this end, the array also has the blue broadcasting lanes (see Fig. 4). With these diagonal lines, columns of PEs always get the features of their

neighbors shifted by 1 pixel. The feature shifter used by the base configuration to implement Kx offsets [see Fig. 5 and Section IV-A1b)] is reused in this configuration to correctly align the features for either 3×3 or 5×5 operation.

The PEs in these rows are then cycle-by-cycle supplied with the Ky subsequent feature rows in their convolution window [see Fig. 4(b)]. This results in three or five rows of partial sums (corresponding to the iterations of the spatially unrolled Fx loop) in these PEs that still need to be accumulated vertically in the PE array.

To support this accumulation, the first row of PEs is configured to accumulate partial sums systolically shifted in (orange in Fig. 4) from the PEs below. This accumulation takes three cycles per channel for a 3×3 depthwise layer and five for a 5×5 . These can be hidden under the next depthwise kernel computation. In regime, the PE rows calculating the depthwise layer are thus used fully, generating an output every three or five cycles.

b) Pointwise part: These outputs are then immediately consumed for a subsequent pointwise (1×1) layer, which is computed in parallel in the remaining PE rows, while the depthwise rows already start processing the next channel. Note that the vertical feature broadcasting wires (red in Fig. 4) are not used by the depthwise rows, so these can be reused in this configuration to send the depthwise results down to the pointwise rows. To keep the pointwise part of the PE fully used until data of the next channel are received from the depthwise layer, each depthwise output should be reused for at least three (resp. five) clock cycles and across all the remaining rows of the PE array. As shown in Fig. 4(c), this is done by temporally reusing the DW output features over multiple sets of pointwise OCs. For optimal utilization, a 3×3 depthwise layer must thus be followed by a pointwise layer with $3 \times (16 - 4) = 36$ OCs (50 for a 5×5) so that all the rows computing the pointwise layer are busy every cycle. Fewer OCs lead to utilization loss in the pointwise rows. For more OCs, the depthwise outputs are not only pipelined to the pointwise PE rows but also saved

in the on-chip memory to be reused later in the remaining 1×1 convolution across the complete array.

Looping as such over different sets of OCs requires these PEs to each have multiple local accumulation registers. Each PE therefore has ten accumulation registers: five to support the pipelined 5×5 DW + PW operation, with double buffering to be able to overlap the next operation with systolically moving outputs out of the array (see below). These registers are added to all PE rows, including the first few, so that they can also provide extra feature reuse in the base configuration to save feature memory bandwidth in that case.

To summarize, Fig. 4(d) brings everything for the DW + PW pipelined configuration together, in which the top PE rows running the depthwise operations are hence pipelined with the bottom PE rows performing the pointwise operations.

B. PE Support for Array Configurations

To support the Conv3D, 3×3 DW + PW, and 5×5 DW + PW PE array configurations, three different types of PEs are used in the array (see Fig. 6) as follows.

- 1) **Type N:** a normal PE with only base functionality to operate for Conv3D.
- 2) **Type D:** a PE with DW convolution support on top of the functionality of normal PEs.
- 3) **Type A:** a PE with support to accumulate passed on psums.

The base functionality present in all three types includes accumulating the product of a vertically broadcasted feature (red in Figs. 4 and 6) and horizontally broadcasted weight (green) in its register file (orange). Requantized outputs are systolically fetched from the PEs through a pipeline formed by the gray registers. PE type N, which has only this base functionality, is used for PE rows 6–15, as these are never part of the DW part of the array.

Type D has an added multiplexer on the feature input so that the PE can use either the diagonally broadcasted feature (blue) when it is part of the DW part of the array configuration or the vertically broadcasted feature (red) when it is not. Furthermore, PEs of type D also make a pipeline to pass on full-precision (32b) psums (orange) to the accumulation row of PEs (type A). This type is used for PE rows 1–6, with either only rows 1–4 using the diagonal feature (blue) for 3×3 DW or all rows 1–6 using it those for 5×5 DW.

Finally, type A, used only for row 0, has an extra multiplexer (orange) on its adder input to support the accumulation of features received from below type D rows.

C. Low Power Array Optimizations

To further reduce the power consumption in the DepFiN accelerator, two changes are made at the implementation level.

- 1) **Zero Gating:** Similar to [19], DepFiN saves activity and therefore dynamic power by not performing multiply-accumulate operations with zero inputs or zero weights. To this end, registers on the feature and weights' inputs of the PEs gate off all accumulation activities if any of the multiplicand operands are zero, saving for instance up to 58% of power if both the operands have 50% sparsity.

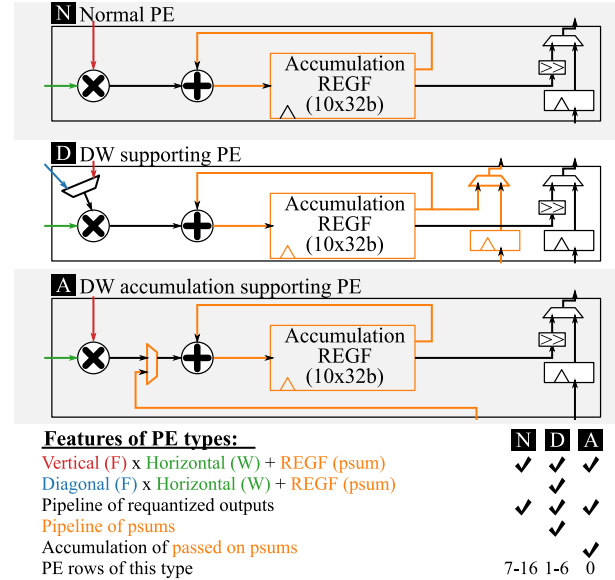


Fig. 6. Three types of PEs together support all PE array configurations of Fig. 4, which shares its color scheme with this figure.

- 2) **Register File Optimization:** (See Fig. 7) After the above optimization, post place-and-route power simulation indicated that 30% of the chip's power consumption went to the flipflops of the PE register files and 2/3rd of this power was due to flipflops of words that were clock-gated and hence functionally inactive, yet had toggling inputs. To save this power, the flipflop's scan enable (SE) is set to *global SE* OR (*NOT write-enable*), which switches the effective input of clock-gated flipflops to the low activity scan inputs (SIs) without harming any logical or (scan) test functionality. P&R simulations showed about 15% savings on the chip's total power because of the reduced internal activity in the flipflops.

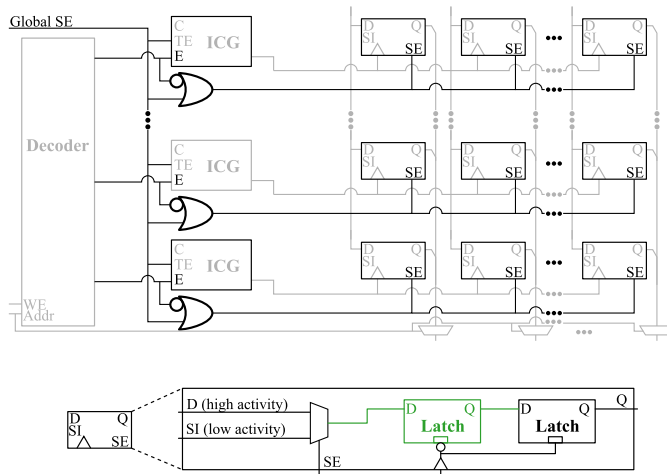
V. DEPTH-FIRST CONTROL FLOW

For a given to-be-executed network, the depth-first compatible control flow of DepFiN is optimized offline in a scheduler and compiler. This can be split into two abstraction levels: 1) creating a schedule of atomic actions (see Table II) on patches and 2) controlling the datapath and memories to do these actions. The latter is done by turning the schedule into a program (off-line) for an on-device μ PC, which configures and starts FSMs that map to the actions (see Table II). These FSMs then do the cycle-by-cycle control of the datapath and memories.

A. DepFiN Scheduler

On the high abstraction level, the FMs are partitioned into patches (see Fig. 8), which are then scheduled into a schedule similar to the one of Fig. 9. To match the PE array's Ox parallelization, the patches are 1×128 pixels large. Scheduling larger patches, e.g. full lines, would be futile as they would still need to be broken into smaller patches matching the PE array at a later control stage, while caching more features.

As mentioned in Section II, the patches can be equally big in all the layers, as seen in Fig. 2. However, as also shown



in Fig. 2, patches computed consecutively across the different layers are not at the same spatial location in different layers: deeper layers *lag* both vertically and horizontally behind the top layers. Accordingly, the partition of the FMs in Fig. 8 shows patch boundaries that horizontally shift to the left for deeper layers (patch width indicators below FMs).

The resulting need of patch alignment across the different layers is supported in hardware by a feature shifter between the PE array and the feature memories. As patches are only 1 pixel high, the vertical lag is handled fully by the schedule.

After partitioning, the algorithm of Listing 1 schedules the patches to be calculated (or loaded from external IO). As per the depth-first paradigm, it always tries to propagate a patch further through the network by computing the next patch in a deeper FM before generating/loading more patches in the same FM, hence the inner *for layer in layers* loop. This results in schedules like the one in Fig. 9, which, for simplicity, only shows when which patch is computed (or loaded for the input) and omits the other actions (see Table II).

Because of the vertical lag, the schedules have three distinct phases: First, an **initialization phase**, in which not enough information is yet present to calculate any patches of the last layer, but an increasing amount of intermediate layers can already execute one or more patches. Then a **regime phase**, in which **every newly loaded input patch leads to one extra patch generated in every layer's FM** (disregarding strides). This phase is regular and can be programmed as a loop. Note, however, that in this loop the layer currently under calculation switches very frequently. Finally, there is the **finalization** phase, in which all information is already present in on-chip memory and the **last output patches can be generated without extra inputs**. For instance, the first blue patch 16 in the finalization phase in Fig. 9 is the first one that does not require an extra red patch to be fetched. From that point on, all the remaining patches in **FM1–FM3 can be generated without extra input (red) patches**. After the schedule is initially generated, it is optimized by rescheduling *load* and *send* actions so that they can be hidden behind computations.

B. Compiler for FSM Control

The actions of the schedule map directly to the hardwired FSMs present in DepFiN. To execute this schedule, these FSMs, however, need to be reconfigured and started many times as every action works on a patch of only 1×128 pixels. This is in contrast to the traditional layer-by-layer accelerators, which need to be configured only once per layer. Because of the many reconfigurations required, it is vital to keep stalls due to reconfiguration to a strict minimum.

To this end, DepFiN uses an on-chip μ PC with the FSM parameters mapped to its register file. While this has the advantage that each configuration parameter and FSM can be updated individually, this implies that updating the configuration of an FSM can take multiple clock cycles. Yet, also these cycles can mostly be hidden underneath compute cycles to prevent stalling of the core.

The compiler takes the schedule of actions as its input, which correspond to the FSM that needs to be started with certain parameters. From this, it generates a program for the microcontroller, containing all the necessary instructions to configure and start the FSMs as meant by the schedule. This compilation happens in three steps. First, the big regime phase loop is detected in the schedule so that it can be compiled as a loop. This is required to prevent overly large programs.

Second, each of the actions in the schedule is compiled into a set of instructions that ensure the parameters are set correctly in the register file and then appends the `copy_params` and `start` commands to copy the parameters to the FSM and then start it. Copying the parameters (all with one instruction) ensures that the μ PC can immediately start overwriting the parameters in its register file in preparation for the next FSM. Depending on which FSM is started and whether the next one can run simultaneously or not, the compiler appends a `wait` instruction to wait for the required amount of cycles to let the FSM finish. The result is a program like the top program depicted in Fig. 10.

Third, in order to prevent stalls, the cycles that are required by the μ PC to prepare the FSM parameters are overlapped with the cycles the FSMs are running. For instance, on benchmark CNN 2 (see below and Fig. 13), these optimizations increased PE utilization by 7%.

VI. MEASUREMENTS AND METRICS

Fig. 11 shows DepFiN’s metrics and die photograph, which was taped out on 7 mm² on GlobalFoundries’ 12LP platform, 4.5 mm² of which is the core area (everything but IO).

A. Peak Electrical Performance

Fig. 12 shows the electrical performance of DepFiN. These results are with the core fully used and fed with random data from the on-chip memories. Worst case results across different kernel sizes (1×1 , 3×3 , or 5×5) and temporal reuse of features (innermost temporal loop of Fig. 4; $1\times$, $3\times$, or $5\times$) are reported. All power except for IO, which is disabled in this experiment, power is included. For Conv3D, several combinations of weight and input activation sparsity are given. This has a large effect on the efficiency, which

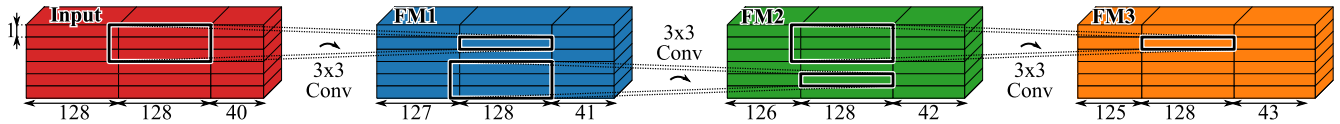


Fig. 8. Partitioning of an FM into patches to be scheduled (thin black lines): patch boundaries shift horizontally across layers to not let output patches require the right neighbors of their corresponding input patches. (Not to scale.)

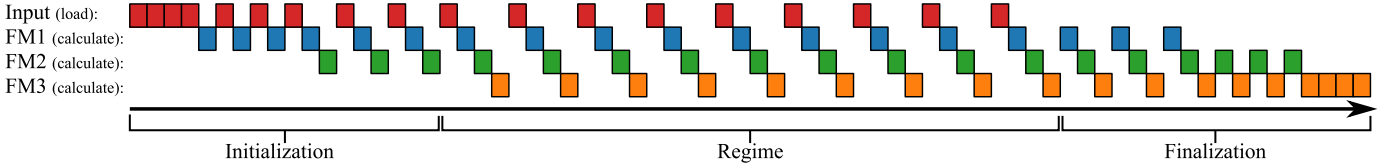


Fig. 9. Schedule of patches of Fig. 8 as made by the algorithm of Listing 1. For the same FM, the patch order is left-to-right and then top-to-bottom. Each layer's next patch is scheduled as soon as its required inputs are loaded/calculated.

```

def schedule(layers, partitioning):
    while True:
        newInputRequired = 1          # assume until proven otherwise

        # find layer that can compute
        for layer in layers:
            if canRun(layer):
                schedule(layer.nextPatch()) # layer can run => schedule it
                newInputRequired = 0       # proven otherwise
            if layer.isNetworkOutput():
                schedule(Send)

        if layers[last].isDone():
            return

        if newInputRequired:
            schedule(Load)

```

Listing 1. Algorithm that creates schedules as in Fig. 8, in which patches are propagated down the network as soon as possible.

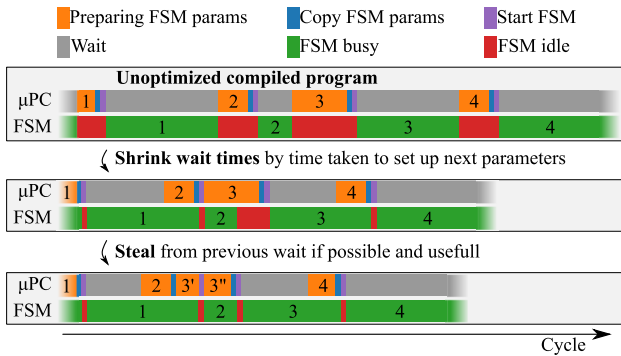


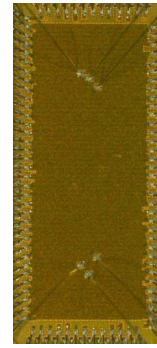
Fig. 10. DepFiN compiler optimization steps hide instructions in FSM runtimes to avoid core stalls and as such increase core utilization.

ranges from 5 to 20 TOPS/W³ at 0.5 V and 250 MHz with the sparsity going from none at all to 75% on both weights and input activations. Also, the DW + PW configuration is included (with no sparsity), which shows only a small loss in efficiency compared with the Conv3D case.

B. CNN Performance

On top of the peak electrical core performance measurements, **DepFiN** is also benchmarked on four multilayer CNNs of various topologies. The benchmarked CNNs are summarized in Fig. 13 and include both the synthetic CNNs and

³1 MAC = 2 OPS in this article.



Process	12nm	
Area	4.5mm ²	excl. IO
	7mm ²	incl. IO
Supply	0.5-0.8V	
Frequency	250-930MHz	
#MAC-units	2048	
SRAM	1056kB	Features
	524kB	Weights
	213kB	Instructions
Precision	8b (32b accumulation)	
Peak performance	3.8TOPs	
Conv3D Energy efficiency	5.0TOPs/W	(dense)
Dwise-pwise Energy efficiency	4.3TOPs/W	(dense)
Area efficiency	0.84TOPs/mm ²	

Fig. 11. Bonded die photograph and chip metrics.

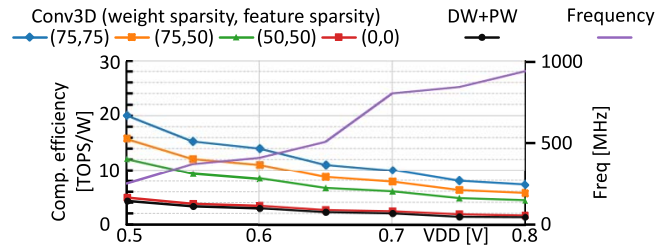


Fig. 12. Peak core performance for Conv3D with different sparsities and DW + PW. (Worst case numbers across different kernel sizes are reported.)

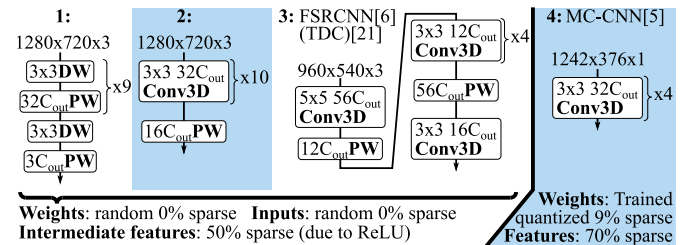


Fig. 13. Overview of the benchmarked CNNs' architectures, inputs, and sparsities. 1 and 2 are synthetic, and 3 and 4 are pixel processing CNNs from literature, respectively, for super-resolution and feature extraction for stereo matching. All CNNs are fully fused for inference on DepFiN.

CNNs from literature and a mix of Conv3D and DW + PW-based networks. Fig. 14 further visualizes the MC-CNN setup as an example. DepFiN chip runs all the four networks completely and with all the layers fused together so that no intermediate features go off-chip for any of these networks, as made possible by our line-buffered 1×128 patch depth-first inference method (see Sections II and V). Table III shows

TABLE III
MEASUREMENTS OF CNNs' (SEE FIG. 13) PERFORMANCE AT
VDD_{CORE} = 0.65

CNN (GMACs)	Freq. [MHz]	P _{core} [mW]	P _{io} [mW]	Util. [%]	Eff. [TOPS/W]
1 (10.1)	310 680	247 530	43 N.A.	41	1.7 2.1
2 (77.7)	190 680	154 527	34 N.A.	84	3.5 4.4
3 (9.7)	170 700	109 407	32 N.A.	72	3.6 4.9
4 (51.9)	440 680	234 361	224 N.A.	93	3.6 7.2

·Output stream enabled|disabled (input always streamed)
·Highest working frequency reported.
·Utilization as (useful MACs) / (2048 x #core clock cycles)

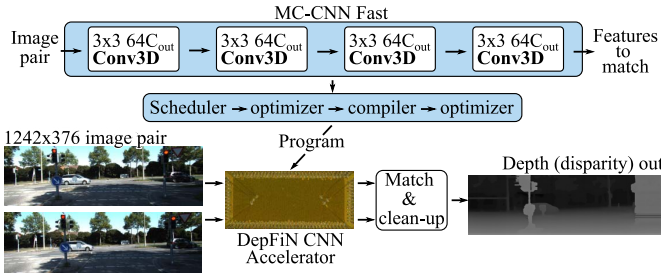


Fig. 14. MC-CNN depth extraction setup: the computationally expensive feature extracting CNN is accelerated on the DepFiN processor, without external memory for intermediate features.

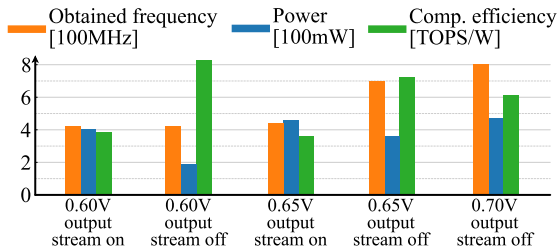


Fig. 15. MC-CNN fast [5] (see Fig. 13) performance on DepFiN.

DepFiN's performance across the CNNs of Fig. 13 at 0.65-V core voltage, while Fig. 15 shows DepFiN's performance on MC-CNN across different core voltages. We report performance numbers with the off-chip CNN output offloading enabled and disabled because of bugs in the feature memory concurrent access arbiter and suboptimal printed circuit board (PCB) design, as these limit the usable bandwidth from DepFiN to the measuring field-programmable gate array (FPGA) in practice by up to 80×. We still do all necessary configurations of the send FSM so that possible stalls due to the μ PC are still accounted for, but do not start the FSM. As the last output patches are still in the feature memory after the program finishes, we can use them for verification as a correctness check. In all the experiments, the inputs are still streamed in as this direction is less hindered by the mentioned problems and is required to run the network, because full FMs are never stored on-chip. The reported IO power includes only the chip side but is surely not an underestimation of the realistic total IO power because: 1) chip measurements showed about 50 pJ/transferred bit, which is high due to the suboptimal PCB design and 2) the outputs of the CNNs are at least as large as the inputs (CNN 1) and usually much larger (CNNs 2–4).

The results show that even on a complete end-to-end network workload, DepFiN can maintain very high utilization and a high TOPS/W taking IO power into account. CNN

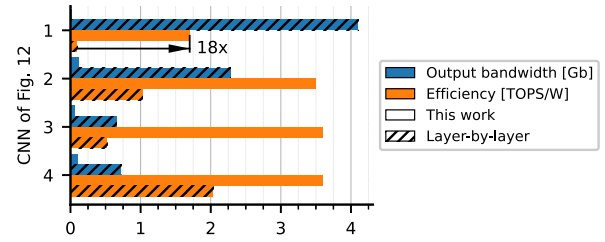


Fig. 16. DepFiN CNN inference efficiency with its line-buffered approach compared with when it would use layer-by-layer inference. For the latter, core energy was assumed equal to the line-buffered approach, the input bandwidth was ignored (as it is not measured for depth-first either, see text) for energy, the optimistic assumptions from [8] (see Section II-C) were used to determine the bandwidth, and a matching 50 pJ per sent output bit was used.

1 does suffer from a lower utilization compared with the other benchmarking networks due to the short runtimes of the DW + PW FSM that make it hard to hide the μ PC configuration setup instructions.

To get a clear view of the gains achieved on DepFiN by its support for the depth-first-like inference, Fig. 16 compares its results with a layer-by-layer reference case (see subscript for details) to show an 18× improvement. The other networks show smaller gain factors, with a minimum of 1.8× for CNN 4, as they have higher computational intensities within layers. These gains, realized solely by the large decrease in external IO bandwidth, show the importance of looking at the full CNN performance, including external IO, and not just a peak core performance. A comparison between the energy efficiency numbers with and without the output stream disabled further shows this importance: the difference is still up to a factor of 2, even though DepFiN requires little more IO than CNNs' initial inputs and final outputs. No intermediate features ever leave the chip for any of these workloads.

The energy efficiency with the output stream disabled is noticeably higher for CNN 4 (MC-CNN-fast) than for the others. This shows the efficiency of the zero-gating, as CNN 4 has more sparsity (see Fig. 13).

VII. RELATED WORK

Alwani et al. [15] use layer fusion based on ROI pyramids on FPGAs. However, as pointed out in Section II and [8], it is not required to keep the full ROI of a final output patch in every layer, while this significantly increased the required on-chip memory capacity. A further difference is that [15] partitions hardware resources to process several layers in a pipelined fashion, whereas DepFiN always uses its full PE array to process one layer's patch to maximize data reuse. Though both are valid methods, partitioning is more suited for FPGAs than ASIPs, as neither dynamically adjusting partition based on the network nor efficient mapping on a fixed partitioning is trivial on ASIPs without significant losses.

The work in [18] is also based on the full ROI pyramids. While caching overlap for one direction of neighboring patches, they use external memory for the other direction. To keep that cost reasonable, the patch size is increased. In contrast, DepFiN does cache all the *alive* intermediate features on-chip, which is made feasible mainly through the

TABLE IV
COMPARISON TABLE

	[15]	[18]	[23]	[17]	[21]	[22]	This work
Process	Virtex-7	65nm	65nm	7nm	40nm	28nm	12nm
Core Area [mm ²]	N.A.	16	6.8	3.04	16	1.9	4.5
Max frequency [MHz]	100	200	200	880	500	470	930
#MAC-units	N.A.	1152	240	2048	4635	N.A.	2048
Layer fusion base method	Pixel ROI-pyramid	Patch ROI-pyramid	Patch ROI-pyramid	N.A. ⁶	Line-by-line DF	Line-by-line DF	line-buffered 1x128 patch DF
SRAM/BMEM [kB]	2509 (F)	572	256	2176	750	206	1780
Peak TOPs	N.A.	0.23 ²	0.644 ³	3.6 ¹	4.65 ¹	1.59 ¹	3.8 ¹
Area efficiency [TOPs/mm ²]	N.A.	0.0145 ²	0.0976 ³	1.18 ¹	0.29 ¹	0.745 ¹⁸	0.84 ¹
Normalized logic area efficiency ⁹ [TOPs/mm ²]	N.A.	0.494 ²	3.28 ³	0.479 ¹	3.56 ¹	5.13 ¹⁸	1.14 ¹
Energy efficiency [TOPs/W]	N.A.	1.9 ²	0.046 / 3.22 ³	6.83 ¹ / 13.3 ¹⁵⁷	5.3 ¹	12.1 ¹⁸	5.0 ¹⁴ 15.8 ¹⁵
Dwise-Pwise E. efficiency [TOPs/W]	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	4.3 ¹⁴
FSRCNN[4] Cache Usage	1050kB ²	113kB ²	N.A.	N.A.	N.A.	N.A.	256kB ¹
540p→1080p BW/inf [MB]	4.2 ²	15.7 ²	N.A.	N.A.	N.A.	N.A.	3.2 ¹
Deepest fusion presented [Conv layers]	5	8	3	2	4	3	20

¹F=8b;W=8b; ²F=16b;W=8b; ³with diluted&deconv 0-skip; ⁴w/o sparsity/dilution; ⁵75% weight and 50% feature sparsity; ⁶save up to 28% DRAM BW for MobileNet by 2-layer fusion; ⁷Fully connected; ⁸some accuracy loss due to predictive early accumulation stop; ⁹equation (1)

realization that not all the features in the ROI pyramid will still be required, but also through a larger on-chip feature memory size.

Lin et al. [17] mention a fusion of two layers at a time to save 28% dynamic random access memory (DRAM) bandwidth. Compared with this, DepFiN fuses far more layers for larger external (DRAM) bandwidth savings.

Min et al. [21] use a similar depth-first [8]-like technique as DepFiN, but works on complete rows at once. This leads to simpler schedulers—there is for instance no horizontal lag to deal with—but requires approximately (*one line-one patch width*) more features to be cached, which results in significant memory overheads for long lines and small kernels. Mo et al. [22] similarly work on full lines and limit the fused layers to one residual block. The latter is also a limitation of [23].

Huang et al. [16] rely on recomputation of features in the overlapped parts of ROI pyramids and thus require a large enough output patch size to avoid too many recomputations (see Section II). They also propose a layer fusion-friendly network architecture, *ERNet*, to further decrease the overhead.

Table IV shows comparison of DepFiN against the above referenced works. While DepFiN has a high peak efficiency, the main highlight is its support for fusing many layers through its line-buffered 128 depth-first approach, as this allows better system efficiency and die area through low external IO bandwidth and on-chip memory usage. As a second highlight, DepFiN's efficiency remains largely intact for DW + PW layer pairs and as such enables the usage of these computationally efficient layers without losing this gained efficiency on the processing system.

Table IV also introduces “normalized logic area efficiency,” which allows to more fairly compare the area efficiency of accelerators by subtracting the memory size from the total core area. The full normalization equation is

$$\frac{\text{TOPs}}{((T_{\text{ref}}[\text{nm}]/T[\text{nm}]))^2 \cdot (A_{\text{core}} - \text{SRAM}[\text{bits}] \cdot A_{\text{bitcell}}))} \quad (1)$$

with T being a technology process and A the die area. For A_{bitcell} , we use data from wikichip.org and assume TSMC and high density if not specified otherwise. We argue that

using deeper or shallower memories is a choice that is mainly orthogonal to the design of the datapath, and hence memory die area should not be included when comparing area efficiency.

VIII. CONCLUSION

This article showed the limitations imposed on efficiency by layer-by-layer network execution (see Section I) and presented a depth-first-based alternative to overcome them (see Section II). By caching only those features that are still needed to compute new features, and propagating 1×128 patches through the network as soon as possible, both good external IO versus on-chip memory capacity trade-offs and parallelization opportunities are guaranteed. Then, a 16×128 PE array using these opportunities was presented (see Section IV). To make the array able to process the computation-friendly yet hard-to-parallelize DW + PW layer pairs with good utilization, the array can be reconfigured at runtime into a novel specialized configuration that pipelines the layers of such pairs to overcome the low computational intensity of depthwise layers. This can be viewed as an extra implementation of the depth-first approach, this time at the PE array level rather than at the processor level.

A control flow capable of handling the challenge introduced by depth-first inference that is frequently switching the active layer is presented (see Section V). It consists of hardwired FSMs for cycle-by-cycle control, configured and started by a built-in μ PC, which is programmed by an offline, depth-first scheduler and offline compiler.

The measurements of the resulting DepFiN CNN processor (see Section VI) show five TOPs/W with dense features and weights at 0.5-V core voltage, and 20 TOPs/W at 0.5 V with 75% sparse features and weights. The difference indicates the efficacy of the implemented zero gating. DepFiN's CNN network efficiency ranges from 2.1 to 7.2 TOPs/W at 0.65 V and excluding external IO, depending on the network's architecture, feature, and weight sparsity. Including IO power, the efficiency ranges from 1.7 to 3.6 TOPs/W at 0.65 V and shows a gain of up to $18\times$ over a layer-by-layer processor with the same on-chip processing efficiency as DepFiN.

ACKNOWLEDGMENT

The authors would like to thank GlobalFoundries, Beijing, China, for the technical discussions and extensive tapeout support on their 12LP platform, with special thanks to Hongqing Zhang and Bingxin Zhang.

REFERENCES

- [1] N.-S. Syu, Y.-S. Chen, and Y.-Y. Chuang, "Learning deep convolutional networks for demosaicing," 2018, *arXiv:1802.03769*.
- [2] C. Ledig et al., "Photo-realistic single image super-resolution using a generative adversarial network," in *Proc. CVPR*, vol. 2, no. 3, Jul. 2017, p. 4.
- [3] N. Friedrich, O. Lobachev, and M. Guthe, "Faking it: Simulating background blur in portrait photography using a coarse depth map estimation from a single image," Univ. West Bohemia, Plzeň, Czechia, Tech. Rep., 2016.
- [4] H. Jiang, D. Sun, V. Jampani, M.-H. Yang, E. Learned-Miller, and J. Kautz, "Super slo-mo: High quality estimation of multiple intermediate frames for video interpolation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 9000–9008.
- [5] W. Luo, A. G. Schwing, and R. Urtasun, "Efficient deep learning for stereo matching," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 5695–5703.
- [6] C. Dong, C. C. Loy, and X. Tang, "Accelerating the super-resolution convolutional neural network," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2016, pp. 391–407.
- [7] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 25, 2012, pp. 1–9.
- [8] K. Goetschalckx and M. Verhelst, "Breaking high-resolution CNN bandwidth barriers with enhanced depth-first execution," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 2, pp. 323–331, Jun. 2019.
- [9] M. Horowitz, "1.1 Computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2014, pp. 10–14.
- [10] JEDEC Committee JC-42.6. *Low Power Double Data Rate 4 LPDDR4 JESD209-4D*. Accessed: Oct. 9, 2022. [Online]. Available: <http://www.jedec.org/standards-documents/results/jesd209-4>
- [11] J. O' Neill, "An overview of neural network compression," 2020, *arXiv:2006.03669*.
- [12] S. Kang, G. Park, S. Kim, S. Kim, D. Han, and H.-J. Yoo, "An overview of sparsity exploitation in CNNs for on-device intelligence with software-hardware cross-layer optimizations," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 11, no. 4, pp. 634–648, Dec. 2021.
- [13] B. Moons, K. Goetschalckx, N. Van Berckelaer, and M. Verhelst, "Minimum energy quantized neural networks," in *Proc. 51st Asilomar Conf. Signals, Syst., Comput.*, Oct. 2017, pp. 1921–1925.
- [14] K. Goetschalckx and M. Verhelst, "DepFiN: A 12 nm, 3.8 TOPs depth-first CNN processor for high res. Image processing," in *Proc. Symp. VLSI Circuits*, Jun. 2021, pp. 1–2.
- [15] M. Alwani, H. Chen, M. Ferdman, and P. Milder, "Fused-layer CNN accelerators," in *Proc. 49th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2016, p. 22.
- [16] C.-T. Huang et al., "ECNN: A block-based and highly-parallel CNN accelerator for edge inference," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchitecture*, Oct. 2019, pp. 182–195.
- [17] C.-H. Lin et al., "7.1 A 3.4-to-13.3 TOPS/W 3.6 TOPS dual-core deep-learning accelerator for versatile AI applications in 7 nm 5G smartphone SoC," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 134–136.
- [18] J. Lee, D. Shin, J. Lee, J. Lee, S. Kang, and H.-J. Yoo, "A full HD 60 fps CNN super resolution processor with selective caching based layer fusion for mobile devices," in *Proc. Symp. VLSI Circuits*, Jun. 2019, pp. C302–C303.
- [19] B. Moons and M. Verhelst, "An energy-efficient precision-scalable ConvNet processor in 40-nm CMOS," *IEEE J. Solid-State Circuits*, vol. 52, no. 4, pp. 903–914, Apr. 2017.
- [20] J.-W. Chang and S.-J. Kang, "Optimizing FPGA-based convolutional neural networks accelerator for image super-resolution," in *Proc. 23rd Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2018, pp. 343–348.
- [21] F. Min et al., "Dadu-eye: A 5.3 TOPS/W, 30 fps/1080p high accuracy stereo vision accelerator," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 10, pp. 4207–4220, Oct. 2021.
- [22] H. Mo et al., "A 12.1 TOPS/W quantized network acceleration processor with effective-weight-based convolution and error-compensation-based prediction," *IEEE J. Solid-State Circuits*, vol. 57, no. 5, pp. 1542–1557, May 2021.
- [23] D. Im, D. Han, S. Choi, S. Kang, and H.-J. Yoo, "DT-CNN: An energy-efficient dilated and transposed convolutional neural network processor for region of interest based image segmentation," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 10, pp. 3471–3483, Oct. 2020.



Koen Goetschalckx was born in Duffel, Belgium, in 1995. He received the B.S. and M.S. degrees (*magna cum laude*) in electrical engineering from KU Leuven, Leuven, Belgium, in 2015 and 2017, respectively, where he is currently pursuing the Ph.D. degree with the Marian Verhelst's Team, ESAT-MICAS, with a focus on embedded and mobile processing of machine learning applications for high-resolution image processing applications.



Fengfeng Wu received the B.S. and Ph.D. degrees in microelectronics and solid-state electronics from Peking University, Beijing, China, in 2008 and 2013, respectively.

He was a member of technical staff with the IBM Design Center, Beijing, from 2013 to 2015. He joined the GlobalFoundries Design Center, Beijing, as a Chip Design Engineer, in 2015. His research interests include digital design and implementation.



Marian Verhelst (Senior Member, IEEE) received the Ph.D. degree from KU Leuven, Leuven, Belgium, in 2008.

She worked as a Research Scientist with Intel Labs, Hillsboro, OR, USA, from 2008 to 2010. She is currently a Full Professor with MICAS Laboratories, KU Leuven, and the Research Director with imec, Leuven. Her research focuses on embedded machine learning, hardware accelerators, HW-algorithm co-design, and low-power edge processing.

Dr. Verhelst was a member of the Young Academy of Belgium and the STEM Advisory Committee to the Flemish Government. She is a member of the Board of Directors of tinyML and active in the 2019 TPCs of DATE, ISSCC, VLSI, and ESSCIRC. She received the Laureate Prize of the Royal Academy of Belgium in 2016, the 2021 Intel Outstanding Researcher Award, and the André Mischke YAE Prize for Science and Policy in 2021. She was the Chair of tinyML2021 and the TPC Co-Chair of AICAS 2020. She is an IEEE SSCS Distinguished Lecturer. She was an Associate Editor of IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS (TVLSI), IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—II: EXPRESS BRIEFS (TCAS-II), and IEEE JOURNAL OF SOLID-STATE CIRCUITS (JSSC).